

ML Sentiment Analysis — End-to-End MLOps Pipeline شرح مشروع

LLMs و MLOps باستخدام تقنيات " (Sentiment Analysis) ده دليل شامل لمشروع "تحليل المشاعر هشرح كل جزء في المشروع: إحنا عملنا إيه، ليه عملناه، واستخدمنا Interview مكتوب بالعربي لمساعدتك في ال إيه، وإزاي بيشتغل.

1. نظرة عامة على المشروع (Project Overview)

Large Language Model (LLM) باستخدام (Sentiment Analysis) المشروع عبارة عن نظام كامل لتحليل المشاعر MLOps لكن نمر بكل مراحل ال، model الفكرة مش بس إننا نبني Qwen2.5-1.5B اسمه (LLM) Model Lifecycle:

- Experimentation: ونسجل النتائج Few-Shot Prompting نجرب استراتيجيات.
 - Optimization: عشان يشتغل أسرع Quantization (GPTQ 4-bit) باستخدام Model نضغط ال. وأخف.
 - Serving: بين النسخة الأصلية A/B Testing ونعمل، FastAPI + vLLM باستخدام Model لل API نوفر.
 - Monitoring: (Data Drift) ونكتشف لو البيانات اتغيرت Production في ال Model نراقب أداء ال.
 - Deployment: بإمان updates عشان ننزل Kubernetes Canary Deployment ننشر بـ.
- مش بس الموديل لازم يكون دقيق، لازم يكون سريع، أخف، قابل Production ليه عملنا كده؟ عشان في ال يكون بأمان بدون ما يوقف الخدمة Deployment للمراقبة، وال.

2. التصميم العام (Architecture)

التصميم العام للمشروع بيتكون من المراحل دي بالترتيب:

1. Training & Few-Shot Experiments: 1-shot، 3-shot، 5-shot prompting، ونستخدم MLflow Tracking.
2. Model في ال (Challenger) ونسخة التحدي (Champion) بنسجل أفضل نسخة: MLflow Registry Registry.
3. Quantization (GPTQ 4-bit): GB 1~ نضغط الموديل من 3~.
4. FastAPI Traffic Router: في Predictions ويسجل ال (80% V1 و 20% V2) بيوزع ال JSONL.
5. vLLM Servers: V1 (FP16) و V2 (GPTQ) ويولدوا ال Requests بيستقبلوا ال Responses.
6. Evidently Drift Detection + Prometheus: في البيانات ويقدرنا بيعتوا Drift بيكتشفوا لو في Alerts.

7. Argo Rollouts (Kubernetes): Canary Deployment (20% → 50% → 80% → 100%) مع Automated Gates.

حسب نسبة معينة، ويسجل V2 ولا V1 ده هو باب الدخول للمستخدم، بيختار إنها يروح لـ Traffic Router الـ عشان نقدر نراقبها بعدين prediction كل

3. (التقنيات المستخدمة) Tech Stack الـ

Infrastructure, Monitoring, Serving, ML, من تقنيات الـ mix إحنا استخدمنا

الفئة	التقنيات
Model	Qwen2.5-1.5B (LLM), Transformers, PyTorch
Quantization	GPTQ, AutoGPTQ, gptqmodel
Experiment Tracking	MLflow
Serving	FastAPI, vLLM, HTTPX
Monitoring	Evidently AI, Prometheus, Grafana
Orchestration	Kubernetes, Argo Rollouts
Infrastructure	Docker, ConfigMaps, ServiceMonitors

عشان PagedAttention بيستخدم optimized serving engine for LLMs ؟ لأنه vLLM فيه اخترنا latency ويقلل الـ throughput يزيده

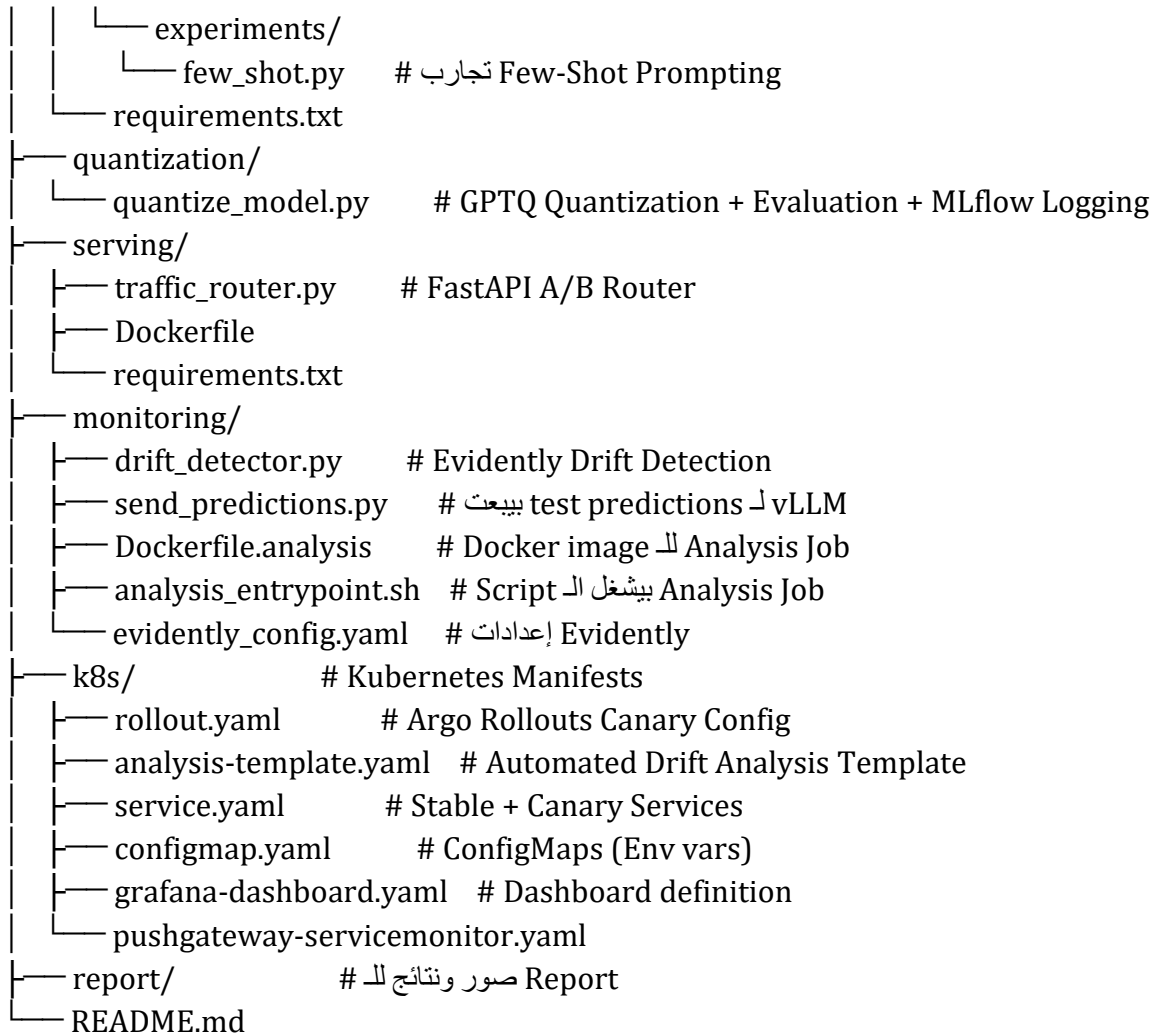
model و experiment tracking لـ standard open-source ؟ لأنه MLflow فيه اخترنا registry، وإدارة runs بين comparison بيسهل،

UI و (HTML) واضحة reports جاهز مع drift detection ؟ لأنه Evidently AI فيه اخترنا service.

4. (Project Structure) هيكل المشروع

واضحة عشان يسهل الصيانة directories المشروع مقسم لـ

```
ml-sentiment-app/
├── data/                # الـ Datasets (train/eval)
├── training/
│   ├── configs/        # experiment_config.yaml (عدادات MLflow و Model)
│   └── src/
│       ├── train.py     # Main script الـ بتاع التدريب والتجارب
│       ├── data_loader.py # datasets تحميل وإنشاء
│       ├── evaluate.py  # Metrics حساب الـ (Accuracy, F1, Precision, Recall)
│       └── registration.py # MLflow Registry تسجيل الموديل في
```



concerns: كل directory واحدة (training vs serving vs monitoring vs deployment). CI/CD لأي حد جديد في الفريق، وبيخلي الـ onboarding ده بييسهل ده لأنه بيحصل أبسط pipeline.

5. البيانات Data الـ

بسيطة CSV files البيانات عبارة عن:

- train_sentiment.csv: بيانات التدريب (fine-tuning لو احتجنا).
- eval_sentiment.csv: بيانات التقييم (Evaluation) — text و label فيها (positive/negative/neutral).

function وفي list of dicts ويحوله لـ CSV بيحمل الـ Data Loader (data_loader.py) الـ create_sample_dataset() dataset بتقدر تعمل testing. صغير بسرعة للـ dataset.

بـ LLM لأننا بنستخدم Traditional Fine-Tuning فيه عملنا كده؟ عشان في المشروع ده مش بنعمل عشان نحسب الأداء Evaluation فالبيانات محتاجة بس للـ (Prompting).

6. Training & Few-Shot Prompting Experiments

إحنا بنعمل 2 حاجات رئيسية، train.py في الـ

أ. Few-Shot Prompting Experiments

عشان prompt في الـ (few-shot) قليلة examples الفكرة: بديل ما ندرّب الموديل من الصفر، بنعطيه يتعلم "المهمة بسرعة".

بيعمل few_shot.py:

- 1-shot: prompt فيه example واحد (مثال positive/negative/neutral).
- 3-shot: 3 examples.
- 5-shot: 5 examples.

كل experiment بيحسب: Accuracy, F1-macro, Precision-macro, Recall-macro, Total Inference Time, Average Latency.

يعني زيادة الـ (5-shot (~75%), 3-shot (~72%), 1-shot-النتائج المشهورة: 1: شوية latency بتحسّن الأداء لكن بتزود الـ examples.

examples عامة، ومع الـ tasks متدربة على Qwen2.5-1.5B زي LLMs ؟ لأن Few-Shot فيه بنعمل fine-tuning بدون (sentiment analysis) المحددة task القليلة بتقدر تتكيف للـ مكلف.

ب. MLflow Integration

عشان MLflow بيستخدم train.py:

- log_param: model_name, n_shots, num_samples.
- log_metric: accuracy, f1_macro, precision_macro, recall_macro, latency.
- log_artifact: classification report (text) و confusion matrix (JSON).

الـ Experiment Name: sentiment-qwen2.5-experiments. الـ Tracking URI: http://localhost:5050.

كان الأفضل، ونرجع run بسهولة، نعرف أي experiments بينا compare؟ عشان نقدر نملّك MLflow فيه reproducibility والـ team work ده مهم جداً في الـ results. أدت لأي parameters لأي

7. Model Registry (MLflow)

MLflow Model Registry بيسجل الموديل في registration.py:

- (few_shot أو quantization_eval) لنوع معين من التجارب (F1 score أعلى) run بيبحت عن أفضل

- registered model جديد في version ك run بييسجل الـ
 - للنسخة المضغوطة (V2) challenger، للنسخة الأصلية (V1) alias: champion يطلق
 - عشان نعرف المسار بتاع كل نسخة tag: vllm_model_path بيضيف
- لو champion نرجع لـ rollbacks (champion vs challenger) A/B testing (champion vs challenger) ده بيخلينا نعمل واضح versioning فشل (، و challenger الـ
- versions: مع 2، team-queens-a2-sentiment: بتلاقي الموديل باسم، MLflow UI في الـ
Version 1 (champion) — V1، و Version 2 (challenger) — V2.

8. Quantization (GPTQ 4-bit)

بيعمل quantization/quantize_model.py

- 1. GPTQ Quantization: weights 16-bit من عشان يضغط gptqmodel library بيستخدم (FP16) 4-bit. model size 3.0~ GB 1.0~ GB (67% reduction).
- 2. Calibration Data: quantization عشان يحسب samples من wikitext dataset بيستخدم scales.
- 3. Post-Quantization Evaluation: eval هـ على evaluate المضغوط وي model بيحمل الـ dataset، accuracy, f1, latency, model size.
- 4. MLflow Logging: artifacts (predictions CSV, confusion matrix PNG) الـ experiment لنفس الـ parameters و metrics يسجل كل

quantization هو (General-purpose Post-Training Quantization) GPTQ؟ GPTQ ليه
method approximate second-order groups بتقسيمها لـ weights بيضغط
information. (صغير ~ 0.78: ~ 0.76 accuracy drop) بيحافظ على دقة الموديل بشكل ملحوظ.
بشكل كبير latency بيقلل الذاكرة والـ

Results:

Model	Accuracy	F1	Size	Latency
V1 (FP16)	~0.78	~0.76	~3.0 GB	~140 ms
V2 (GPTQ 4-bit)	~0.76	~0.74	~1.0 GB	~90 ms

بيفرق مع الـ latency غالي، والـ GPU memory، Production؟ في الـ quantization ليه عملنا
تقريباً quality أضعف، وببيستجيب أسرع، وبنفس الـ GPUs بيقدّر يشتغل على الـ 4-bit model user.

9. Serving & A/B Testing (FastAPI Traffic Router)

serving/traffic_router.py عبارة عن FastAPI app:

أ. Endpoints

- /health: Health check.

- /predict: ويرجع {label, model_version, latency_ms} text (JSON) بيستقبل.
- /logs: prediction logs (JSONL) بيرجع.
- /stats: بيرجع A/B statistics (counts, percentages, average latencies).

ب. Traffic Split

requests يعني 80% من الـ (environment variable ممكن تتحكم فيه من) $V1_WEIGHT = 0.80$ و $V2$ (quantized/challenger) و 20% لـ $V1$ (original/champion) بيروحو الـ

vLLM لـ request وبعدين بيعت الـ version لاختيار الـ random.random() بيستخدم Router الـ httpx.AsyncClient باستخدام ($V1_URL$ أو $V2_URL$) المناسب server.

ج. Prompt Template

Prompt ثابت الـ "Classify the sentiment of the following text as positive, negative, or neutral.\nText: {text}\nSentiment:". consistency ده بيضمن $V1$ و $V2$.

د. Logging

prediction كل JSONL file: timestamp, model_version, input_text, predicted_label, latency_ms. الـ logging ده أساسى لـ Drift Detection, Debugging, Audit.

automatic وبيقدم (httpx مع async/await بيستخدم) asynchronous، ؟ لأنه سريع FastAPI ليه documentation (Swagger UI). A/B Testing الـ وليه. $V2$ (quantized) ؟ عشان نختبر الـ Production ولو نجح نزود النسبة، users بشكل آمن مع نسبة قليلة من الـ

10. Monitoring & Drift Detection (Evidently AI + Prometheus)

بيعمل monitoring/drift_detector.py:

- current predictions (canary/new) مع reference predictions (baseline) بيقرن.
- text length, latency, prediction distribution عشان يحسب Evidently AI DataDriftPreset بيستخدم.
- ويوفره محلياً (drift_report.html) HTML report بيطلع.
- Evidently UI Service (RemoteWorkspace) لـ report مفعّل، بيعت الـ --push لو.
- Prometheus metric كـ drift_score مفعّل، بيعت الـ --pushgateway-url لو (sentiment_drift_score) لـ Pushgateway.

JSONL كـ responses ويسجل الـ vLLM endpoint لـ test texts بيعت الـ send_predictions.py، reference و current datasets عشان نقدر نيني.

evidently_config.yaml: بيحدد: check_interval (300s)، min_samples (50)، reference_window (500)، current_window (100)، numerical_method (Kolmogorov-Smirnov test)، categorical_method (Chi-squared test).

concept drift) بتتغير مع الوقت users بيانات الـ Production مهمة؟ لأن في الـ Drift Detection فيه clothes، reviews عن وجايه electronics عن reviews لو الموديل اتدرب على (data drift أو drift). عشان بدرى إن في تغير Evidently. الأداء هيقبل.

Grafana dashboards و time-series metrics ؟ عشان يقدم Prometheus Alertmanager. الـ Pushgateway الـ Analysis Job مهم لأن الـ batch job (batch job) بيشتغل لفترة قصيرة مش continuous service.

11. Kubernetes Deployment (Canary with Argo Rollouts)

progressive delivery: عشان Kubernetes + Argo Rollouts المشروع بيستخدم

أ. Manifests في k8s/

- rollout.yaml: الـ Rollout resource (بدل Deployment) فيه spec.strategy.canary steps: بيحدد
 - 20% traffic → Drift Analysis Gate (analysis-template)
 - 50% traffic → Pause 30s
 - 80% traffic → Pause 30s
 - 100% traffic → Rollout complete
- service.yaml: 2 Services (sentiment-model-stable و sentiment-model-canary). الـ traffic عشان يوجه selector labels الـ manage بي Argo Rollouts stable أو canary pods.
- analysis-template.yaml: الـ drift-analysis اسمه AnalysisTemplate بيحدد (analysis_job) بيستخدم image: registry.local:5000/sentiment-analysis-job:latest. الـ Job بيشتغل drift-score-check: تانية metric وفي drift_detector.py و send_predictions.py بيشتغل Job Abort. بيبت Rollout لو فشل، الـ $drift_score < 0.5$ لو Prometheus بتسأل
- configmap.yaml: بيحتوي env vars: MLFLOW_TRACKING_URI, EVIDENTLY_URL, PROMETHEUS_PUSHGATEWAY_URL.
- grafana-dashboard.yaml: الـ Drift Score definition JSON بيحتوي ConfigMap
- pushgateway-servicemonitor.yaml: الـ Prometheus عشان ServiceMonitor يكتشف Pushgateway.

ب. Flow بتاع Canary Deployment

1. kubectly apply -f k8s/ بنعمل
2. (V2/canary) على النسخة الجديدة pods بيحط 20% من الـ Rollout الـ
3. الـ Analysis Template Job: بيشتغل predictions الـ baseline و canary، يحسب drift، بيشتغل Prometheus الـ push metric
4. الـ drift-score-check Prometheus: هل $sentiment_drift_score < 0.5$ ؟

stable. % ويرجع لـ Abort 100: بيكمل لـ 50 → 80% → 100%. لو لا Rollout لو نعم: الـ 5.

بس، مش users أو بتقل الأداء، بيأثر على 20% من الـ bug ؟ عشان لو النسخة الجديدة فيها Canary ليه
data-driven يكون decision ؟ عشان الـ Automated Analysis Gates وليه. blast radius ده بيقلل الـ 100%.
manual مش driven.

12. Docker & Containerization

serving/Dockerfile: multi-stage build: بيستخدم:

- Stage 1 (builder): dependencies بيثبت في /install.
- Stage 2: non-root user (appuser) بيشتغل بـ traffic_router.py، بيضيف /install بياخد security. لـ

monitoring/Dockerfile.analysis: evidently, pandas, scikit-learn, httpx, prometheus-client. بيستخدم analysis_entrypoint.sh كـ endpoint.

final في الـ build tools يكون صغير (مش بنحتاج image size ؟ عشان Multi-stage build ليه
containers. في security best practice ؟ Non-root user وليه). image).

13. Key Takeaways النتائج والـ

Few-Shot Performance:

Experiment	Accuracy	F1 Score	Latency (ms)
1-shot	~0.65	~0.62	~120
3-shot	~0.72	~0.70	~145
5-shot	~0.75	~0.73	~160

Quantization vs Baseline:

Model	Accuracy	F1	Size	Latency
V1 (FP16)	~0.78	~0.76	~3.0 GB	~140 ms
V2 (GPTQ 4-bit)	~0.76	~0.74	~1.0 GB	~90 ms

Key Takeaways:

- fine-tuning. بيحسن الأداء بشكل ملحوظ بدون Few-Shot Prompting.
- GPTQ 4-bit Quantization size بيقلل 67% و latency بـ 35% مع drop بـ صغير في الـ accuracy.
- A/B Testing + Traffic Routing بييسمح بـ safe rollout لـ الجديدة models.
- Drift Detection + Prometheus + Grafana = Monitoring شبه real-time.
- Canary Deployments + Automated Gates = Deployment و data-driven بأمان.

- MLflow Tracking + Registry = Reproducibility و Version Management.

14. وإزاي تجاوب عليها Interview أسئلة متوقعة في ال

"Fine-Tuning و Few-Shot Prompting س1: إيه الفرق بين

الموديل. سريع، مش محتاج weights بدون تغيير prompt في ال examples بيستخدم Few-Shot ج: أقوى، وقت GPU، أكثر data محتاج، weights بيغير Fine-Tuning. بسيطة tasks قوي، ومناسب لـ GPU Sentiment Analysis لأن Few-Shot معقدة. في المشروع، اخترنا أطول، لكن بيحسن الأداء أكثر على task (1.5B parameters) صغير model بسيطة وال task.

"AWQ أو INT8 مش GPTQ س2: إيه استخدمتو

بيحافظ أكثر على الدقة INT8. مع دقة محتفظة بشكل كويس (4-bit) أعلى compression بيقدّم GPTQ ج: Qwen أحسن للـ documentation أشهر و GPTQ جديد وكويس بس AWQ. أقل compression بس models. accuracy و size reduction بين balance لأنه GPTQ اخترنا.

"Drift Detection س3: إزاي بيشتغل الـ

current dataset مع reference dataset (baseline) عشان يقارن Evidently AI ج: بيستخدم numerical features (KS test) و categorical features (Chi-squared) للـ drift score. drift detected، يعتبر score > 0.5 لو الـ Prometheus بيتبعث لـ score. gate بيستخدمه كـ Argo Rollouts، و Grafana، و Pushgateway.

"Blue-Green و Canary Deployment س4: إيه الفرق بين

الـ switch وبي (blue=old, green=new) كاملين environments بيشتغل 2 ج: Blue-Green: traffic (النسخة الجديدة) مثلاً 20% ويزودها تدريجياً traffic بيودي نسبة صغيرة من الـ Canary: دفعة واحدة traffic. users. استخدمنا. أمن لأن لو في مشكلة بيأثر على قليل من الـ Canary Rollouts.

"normal scraping مش Prometheus Pushgateway س5: إيه استخدمتو

الـ continuous service مش (Kubernetes Job) batch job بيشتغل كـ Analysis Job ج: لأن الـ batch jobs بيسمح للـ Pushgateway. تشغيل دائماً endpoint بيحتاج Prometheus normal scraping. مرة واحدة metrics إنها تبعث.

"Traffic Router س6: إزاي بيشتغل الـ

weight حسب (V1 أو V2) version بيختار. /predict على request بيستقبل FastAPI app ج: sentiment الـ parse بي، response المناسب. بيستقبل الـ vLLM server لـ request بيبيعت الـ (80/20). client. ويرجع للـ JSONL، بي سجل في، label.

"MLflow Model Registry س7: إزاي بيشتغل الـ

run كـ version الـ بي سجل الـ experiment. في (F1 أعلى) run بي بحث عن أفضل registration.py ج: tag (vllm_model_path) بيضيف (champion/challenger) alias بيطلق registered model. في rollback والـ A/B testing عشان نعرف الموديل الفعلي. ده بي سهل

15. ملخص سريع (Executive Summary)

End-to-End MLOps Pipeline لـ Sentiment Analysis LLM: المشروع بيظهر

- MLflow بالـ track ون (1/3/5-shot) Few-Shot Prompting مع Qwen2.5-1.5B بنستخدم
- الأداء evaluate ون (3x smaller) GPTQ 4-bit بنضغط الموديل بـ
- FastAPI Traffic Router بـ API بنقدم A/B Testing (80/20 split).
- Data Drift بـ Evidently AI و Prometheus بنراقب
- Kubernetes Canary Deployments باستخدام Argo Rollouts + Automated Gates بننشر بـ

تدريجي deployment، optimization، بأمان، مراقبة Production في LLM الـ Goal: